



WACHEMO UNIVERSITY

COLLEGE OF ENGINEERING AND TECHNOLOGY SCHOOL OF COMPATIC
AND INFORMATICS DEPARTEMENT OF INFORMATION TECHNOLOGY

PROJECT COURSE : DATA STRUCTURE AND ALGORITHM

Group name:

ID

1, Henok Brihanu.....	176534
2, Meron Kadre.....	179181
3, Tadele Tesfaye.....	17D2795
4, Zarhun Keton.....	17D5880

Submitted to : Mr. Tsagaye
Submission Date may/2026

Project title

Bank Customer Queue System

Simulate a customer queue in a bank using a singly linked list. Customers can be added (enqueue) and removed (dequeue) based on service completion.

Introduction

The project is depend worked in single linked list and in Queue.

The customers is managed in ticket number because this is very important way to manage customers

Project Overview

This project is a Bank Ticket Management System designed to handle customer queues efficiently. It uses the Singly Linked List data structure to manage waitlists in real-time, allowing multiple bank branches (like CBE, Abyssinia, and Awash) to operate their own separate queues.

Core Data Structure: Singly Linked List

Instead of using a static Array, we used a **Linked List** for several professional reasons:

Dynamic Size: The number of customers is not fixed; the list grows as people arrive and shrinks as they are served.

Memory Efficiency: Memory is allocated only when a new customer arrives and is freed immediately after they are served.

The FIFO Principle (First-In-First-Out)

The system strictly follows the **FIFO** logic, which is the foundation of a **Queue**.

Enqueue (Adding): Performed at the **Rear** of the list.

Dequeue (Serving): Performed at the **Front** of the list.

Object-Oriented Programming (OOP) Approach

The system uses **Classes and Objects**. Each bank branch is an **Object** of the `BankQueue` class. This allows:

Encapsulation: Each bank's data (waitlist) is hidden and protected from other banks.

Reusability: One class can create unlimited bank branches.

Technical Operations

Node Creation: Each customer is a "Node" containing a Name, Account Number, Ticket Number, and a Pointer to the next person.

Traversing: To show the waitlist, the program starts from the `front` pointer and moves through each node until it reaches `NULL`.

Memory Management: We use the `delete` keyword to prevent **Memory Leaks**, ensuring the program remains fast and stable.

The algorithm code

```
#include <iostream>
#include <string>

using namespace std;

// 1. DATA STRUCTURE: Node representing a Customer
struct Customer {
    string name;
    string accountNumber;
    int ticketNumber;
    Customer* next;
};

// 2. CLASS: Managing the queue logic for each bank
class BankQueue {
private:
    Customer *front, *rear;
    int nextTicket;
    string bankName;

public:
    BankQueue(string name) {
        front = rear = NULL;
        nextTicket = 1;
        bankName = name;
    }

    void showWelcome() {
        cout << " WELCOME TO " << bankName << endl;
    }
}
```

```

void registerCustomer(string cName, string accNo) {
    Customer* newNode = new Customer();
    newNode->name = cName;
    newNode->accountNumber = accNo;
    newNode->ticketNumber = nextTicket++;
    newNode->next = NULL;

    if (rear == NULL) {
        front = rear = newNode;
    } else {
        rear->next = newNode;
        rear = newNode;
    }
    cout << "\n[!] Success: Ticket #" << newNode->ticketNumber
        << " issued for " << cName << " at " << bankName << endl;
}

void serveNext() {
    if (front == NULL) {
        cout << "\n[!] No customers in " << bankName << " queue." << endl;
        return;
    }
    Customer* temp = front;
    cout << "\n>>> NOW SERVING: Ticket #" << temp->ticketNumber
        << " (Name: " << temp->name << ")" << endl;

    front = front->next;
    if (front == NULL) rear = NULL;
    delete temp;
}

void display() {
    cout << "\n--- " << bankName << " WAITLIST ---" << endl;
    if (front == NULL) {
        cout << "Queue is empty." << endl;
        return;
    }
    Customer* curr = front;
    while (curr != NULL) {
        cout << "[#" << curr->ticketNumber << " | " << curr->name << "]" -> " ";
        curr = curr->next;
    }
    cout << "END" << endl;
}
};

```

// 3. VALIDATION FUNCTIONS (LOOP-FREE VERSION)

// Checks if name is empty OR contains numbers

```
bool isNameValid(string name) {  
    if (name.empty()) return false; // Validation requirement 1  
  
    // string::find_first_of checks if any character from "0123456789" exists  
    if (name.find_first_of("0123456789") != string::npos) {  
        return false; // Found a number  
    }  
    return true;  
}
```

// Checks if account is empty OR contains letters/symbols

```
bool isAccountValid(string acc) {  
    if (acc.empty()) return false; // Validation requirement 2  
  
    // string::find_first_not_of checks if any character is NOT "0123456789"  
    if (acc.find_first_not_of("0123456789") != string::npos) {  
        return false; // Found a non-digit  
    }  
    return true;  
}
```

// 4. MAIN INTERFACE

```
int main() {  
    BankQueue cbe("CBE"), abyssinia("ABYSSINIA"), awash("AWASH"), dashen("DASHEN"),  
    nib("NIB");  
  
    int bankChoice, actionChoice;  
    string name, acc;  
  
    while (true) {  
        cout << "\n MultiBANK CUSTOMER QUEUE SYSTEM " << endl;  
        cout << "Select Bank:\n1. CBE\n2. Abyssinia\n3. Awash\n4. Dashen\n5. Nib\n6. Exit\nChoice:"  
        ";  
  
        if (!(cin >> bankChoice)) {  
            cout << "[Error] Please enter a number." << endl;  
            cin.clear();  
            cin.ignore(1000, '\n');  
            continue;  
        }  
  
        if (bankChoice == 6) break;  
        if (bankChoice < 1 || bankChoice > 5) {  
            cout << "[Error] Invalid selection!" << endl;  
        }  
    }  
}
```

```

    continue;
}

BankQueue* selectedBank;
if (bankChoice == 1) selectedBank = &cbe;
else if (bankChoice == 2) selectedBank = &abyssinia;
else if (bankChoice == 3) selectedBank = &awash;
else if (bankChoice == 4) selectedBank = &dashen;
else selectedBank = &nib;

selectedBank->showWelcome();

cout << "Action:\n1. Get Ticket\n2. Serve Next\n3. Show Waitlist\nChoice: ";
cin >> actionChoice;

switch (actionChoice) {
    case 1:
        cin.ignore(1000, '\n'); // Clear buffer properly

        cout << "Enter Customer Name: ";
        getline(cin, name);
        if (!isNameValid(name)) {
            cout << "[Error] Invalid Name! (Cannot be empty or contain numbers)" << endl;
            break;
        }

        cout << "Enter Account Number: ";
        getline(cin, acc);
        if (!isAccountValid(acc)) {
            cout << "[Error] Invalid Account! (Must be digits only)" << endl;
            break;
        }

        selectedBank->registerCustomer(name, acc);
        break;

    case 2:
        selectedBank->serveNext();
        break;

    case 3:
        selectedBank->display();
        break;

    default:
        cout << "[Error] Invalid action choice!" << endl;
}

```

```
}  
  
cout << "\nSystem shutdown. Have a great day!" << endl;  
return 0;  
}
```